Universidad Nacional Autónoma de México

Faculty of Engineering

Electrical Engineering Division

Course: Compilers **Group:** 5

Semester: 2026-2

Report:
Lexical Analyzer (Lexer)

Professor:

M.C. René Adrián Dávila Pérez

Student IDs:

320060829

320317264

320206968

320025561

320266083

Delivery Date:

Sunday, March 15th, 2026

Contents

1 Introduction **2**

2 Theorical Framework **2**
2.1 Lexical Analysis (Scanner / Lexer) 3
2.2 Syntactic analysis (Parsing) and CFG (CONTEXT-FREE GRAMMARS) . . 3
2.3 Parsing Strategies: Top-Down Approach and Precedence Resolution 4

3 Desarrollo **4**
3.1 Descripción general de la implementación 4
3.2 Gramatica 5
3.3 Entrada/Salida y Ejecución 13
3.4 Relación con la CFG propuesta 13
3.5 Pruebas realizadas 14

4 Resultados **14**

5 Conclusiones **14**

Referencias **15**

1 Introduction

This document contains the necessary documentation for the understanding in the construction of a lexical analyzer, where it will process source code by reading it from a file or text string. This program will be able to identify, classify and count the tokens in the code that was provided to it, in addition these will be separated into categories such as keywords, identifiers, operators, punctuators, constants and literals.
However, the lexer program meets the restriction of not making use of the FLEX tool, so a lexical analyzer was implemented using the Python Pygments library, which provides a lexer for the C language.
Now then, the lexical analyzer represents the initial phase of a compilation process, where this sets the foundations for the formulation and improvement of a context-free grammar (CFG), this will use a left factorization, which later will be used for the construction of a parser, which will be responsible for validating the syntactic structure of the code that we want to compile, ensuring that it complies with the rules defined in the CFG.

Objectives

- Build a lexical analyzer capable of processing code through the passing of a file or a text string.
- Generate a table with the lexical analysis of the input, classifying the tokens (such as keywords, identifiers, operators, punctuators, constants and literals) and giving a count for each category.
- Provide a basic proposal of Context-Free Grammar (CFG) that represents one of the reading examples, applying left factorization to improve that grammar.
- Package the application following software engineering practices, name it as `unam.fi.compilers.g5.00` and share the repository link with public permissions.

2 Theorical Framework

Firstly, the traduction of a high level programming languaje into a low level one is a process structured in multiple phases. [1]
In this proyect we will aboard the inicial stages of the front-end of the compiler: the lexical analysis and the syntatic analysis. This phases work together in order to validate that the source code given by the user is well formed before proceding with the semantic evaluation or the code generation. In this case, the validation procees its aplied to a code that is written in C that uyses a specific set of instructions

2.1 Lexical Analysis (Scanner / Lexer)

The lexical analysis represents the first stage of the languaje processing. Its fundamental responsability is to redd the code character by character and group them in secuences with logical meaning, known as lexemes.[1]
Starting from this lexemes, the analyzer, generatees a secuence of tokens (as identifiers, key words, constants, special characters, operators, punctuation, literals). In the proyect, the lexical analyzer identifies specific sets as:

- **ID:** Identifiers used to give name to the variables
- **VALUE_NUM:** Literals that represent any number whole and decimal
- **VALUE_CHAR:** Literals that represent a unique character enclosed between simple commas
- **TEXT_STRING:** Text strings enclosed in double quotes, mostly used as format specifiers

2.2 Syntactic analysis (Parsing) and CFG (CONTEXT-FREE GRAMMARS)

The syntatic analyzer is the second stage in this compiler. It recibes the flow of tokens generate by the lexical analizer and verifies that the secuence fullfill the structural rules of the language. [1]
To define this rules in a rigurous way, it uses the CFG (CONTEXT-FREE GRAMMARS), it helps to stablish which rules are valid using the production rules.[1]
In this proyect, the CFG defines the structure to prodecue any instruction (StmtList), covering numeric declarations (DeclNum), assignments (AssignSmt), and flow control statements (lfStmt, WhileStmt, ForStmtm)
An example given is the formal rule of declaring a number (and optionally initialize it) it defines matematically in the next way:

$$DeclNum- > TypeNumIDAssignOpNum$$

The main objective of the parser is to organize the sequence of tokens into a hierarchical structure called a parse tree (or syntax tree). This structure shows how the tokens relate to each other according to the language rules and confirms that the code follows the correct syntax [2]

2.3 Parsing Strategies: Top-Down Approach and Precedence Resolution

There are two main ways to construct the syntax tree: Top-Down and Bottom-Up. This project implements a Top-Down approach through a Recursive Descent Parser. In this method, the tree is built starting from the initial symbol of the grammar down to the leaves (the tokens), associating a recursive function with each grammar rule to process the input from left to right. [2]
To ensure that this approach works mathematically without entering infinite loops and respecting the operation hierarchies, the grammar was designed with specific features:

- **Elimination of left recursion (Left-Recursion):** The production rules for mathematical expressions (MathExpr) were rewritten to be right-recursive.
- **Layered Precedence:** To evaluate logical conditions (Cond) while respecting the standardized hierarchy, the rules were divided into levels. The analysis starts at the lowest priority level looking for the OR logical operator (CondOrPrime), descends to the intermediate AND level (CondAndPrime), evaluates the level of relational comparisons (CondRel), and culminates at the highest priority level processing the logical NOT and parentheses (CondUnary).

3 Desarrollo

3.1 Descripción general de la implementación

La implementación del Lexer esta escrita en Python, pero nuestro CFG considera código escrito en C con las instrucciones descritas en SmtList y nos apoyamos de pygments de lex para analizar lineas de entrada ya sea mediante teclado o por medio de un archivo.
Al momento de usar Pygments se tuvó en cuenta que la Familias de tokens de Pygments no coinciden exactamente con los tokens vistos en el curso, por lo que la asociación quedaría de la siguiente menera:

Familia de tokens (Pygments)	Categoría del Curso
Token.Keyword	KEYWORD
Token.Name	IDENTIFIER
Token.Operator	OPERATOR
Token.Punctuation	PUNCTUATION
Token.Literal.Number	CONSTANT
Token.Literal / Token.Literal.String	LITERAL

Table 1: Mapeo de tokens de Pygments a las categorías del curso.

Cabe recalcar que se consideraron los casos especiales de los literales de cadena delimitados por comillas en una misma línea, agrupando la comilla de apertura, el contenido y la comilla de cierre en un único token para que se reconozca como un literal.
Al final el programa después de procesar la entrada deberá imprimir una tabla con el análisis léxico de la entrada y el conteo de los grupos de tokens.

3.2 Gramatica

1. Reglas de producción

$$
\begin{aligned}
\text{Stmt} &\rightarrow \text{Stmt StmtList} \mid \varepsilon \\
\text{StmtList} &\rightarrow \text{DeclNum} \mid \text{DeclChar} \mid \text{AssignStmt} \mid \text{IfStmt} \mid \text{WhileStmt} \\
&\quad \mid \text{ForStmt} \mid \text{PrintStmt} \mid \text{ScanStmt} \mid \text{UnaryStmt}
\end{aligned}
$$

Donde:

- **Stmt** es el Símbolo Inicial para generar instrucciones.
- **StmtList** representa las instrucciones específicas que se pueden producir.
- **DeclNum** y **DeclChar** son las declaraciones de números y caracteres.
- **AssignStmt** es para la asignación de variables.
- **IfStmt, WhileStmt, ForStmt** son las estructuras de control de flujo.
- **PrintStmt, ScanStmt** son las funciones de entrada y salida de datos.
- **UnaryStmt** es para las sentencias unarias (pre/post incrementos).

2. Reglas de produccion:

$$
\begin{aligned}
\text{DeclNum} &\rightarrow \text{TypeNum ID AssignOpNum;} \\
\text{AssignOpNum} &\rightarrow \text{MathExpr} \mid \varepsilon \\
\text{DeclChar} &\rightarrow \text{TypeChar ID AssignOpChar;} \\
\text{AssignOpChar} &\rightarrow \text{VALUE_CHAR} \mid \varepsilon \\
\text{AssignStmt} &\rightarrow \text{ID AssignExprOp MathExpr} \mid \text{ID VALUE_CHAR;} \\
\text{TypeNum} &\rightarrow \text{int} \mid \text{float} \mid \text{double} \\
\text{TypeChar} &\rightarrow \text{char}
\end{aligned}
$$

Donde:

- DeclNum : Es la PR para declarar un número o declarar un número e inicializarlo.
- DeclChar : PR para declarar un carácter o carácter un número e inicializarlo.
- AssignStmt : Es la PR para asignar el valor a un dato ya declarado anteriormente.
- TypeNum : Los tipos de datos numérico que acepta.
- TypeChar : Tipo de dato de carácter que acepta.

3. Aplicando factorización por la izquierda:
La declaración de variables $DeclNum \rightarrow TypeNum\ ID\ ; \mid TypeNum\ ID\ =\ MathExpr\ ;$ tiene el factor común $TypeNum\ ID$, por lo que podemos expresarla como:

$$\begin{aligned} \text{DeclNum} &\rightarrow \text{TypeNum ID AssignOpNum;} \\ \text{AssignOpNum} &\rightarrow \text{MathExpr} \mid \varepsilon \end{aligned}$$

También la expresión para operaciones continuas

$$\text{MathExpr} \rightarrow \text{MathExpr MathOp MathOperand} \mid \text{MathOperand}$$

presentaba recursividad por la izquierda, por lo que fue factorizada (recursividad por la derecha) de la siguiente manera:

$$\begin{aligned} \text{MathExpr} \rightarrow\ & \text{MathOperand} \\ & \mid \text{MathOperand MathOp MathExpr} \end{aligned}$$

Este procedimiento ayudará a evitar el no determinismo, también con la ayuda de la recursividad por la derecha ya realizada, donde tenemos elementos no terminales en el lado derecho, por lo que esto puede ayudar con las cadenas repetitivas y recursivas que podemos hacer al anidar operaciones o llamadas lógicas.

Finalmente la gramática está definida por las expresiones:

$$
\begin{aligned}
\text{Stmt} &\rightarrow \text{Stmt StmtList} \mid \varepsilon \\
\text{StmtList} &\rightarrow \text{DeclNum} \mid \text{DeclChar} \mid \text{AssignStmt} \mid \text{IfStmt} \mid \text{WhileStmt} \mid \\
&\quad \text{ForStmt} \mid \text{PrintStmt} \mid \text{ScanStmt} \mid \text{UnaryStmt} \\
\text{DeclNum} &\rightarrow \text{TypeNum ID AssignOpNum;} \\
\text{AssignOpNum} &\rightarrow \text{MathExpr} \mid \varepsilon \\
\text{DeclChar} &\rightarrow \text{TypeChar ID AssignOpChar;} \\
\text{AssignOpChar} &\rightarrow \text{VALUE_CHAR} \mid \varepsilon \\
\text{AssignStmt} &\rightarrow \text{ID AssignExprOp MathExpr} \mid \text{ID VALUE_CHAR;} \\
\text{TypeNum} &\rightarrow \text{int} \mid \text{float} \mid \text{double} \\
\text{TypeChar} &\rightarrow \text{char} \\
\text{IfStmt} &\rightarrow \text{if (Cond) \{StmtList\}} \\
\text{WhileStmt} &\rightarrow \text{while (Cond) \{StmtList\}} \\
\text{ForStmt} &\rightarrow \text{for (ForInit; ForCond; ForStep) \{StmtList\}} \\
\text{ForInit} &\rightarrow \text{TypeNum ID AssignOpNum} \mid \text{ID AssignExprOp MathExpr} \mid \varepsilon \\
\text{ForCond} &\rightarrow \text{Cond} \mid \varepsilon \\
\text{ForStep} &\rightarrow \text{IncDecOp ID} \mid \text{ID IncDecOp} \mid \\
&\quad \text{ID AssignExprOp MathExpr} \mid \varepsilon \\
\text{PrintStmt} &\rightarrow \text{printf(PrintArgs);} \\
\text{PrintArgs} &\rightarrow \text{TEXT_STRING} \mid \text{TEXT_STRING, IdList;} \\
\text{ScanStmt} &\rightarrow \text{scanf(TEXT_STRING, \&ID);} \\
\text{IdList} &\rightarrow \text{ID} \mid \text{ID, IdList} \\
\text{UnaryStmt} &\rightarrow \text{IncDecOp ID;} \mid \text{ID IncDecOp;} \\
\text{IncDecOp} &\rightarrow ++ \mid - \\
\text{AssignExprOp} &\rightarrow = \mid += \mid \text{-=} \mid /= \\
\text{MathExpr} &\rightarrow \text{MathOperand} \mid \text{MathOperand MathOp MathExpr} \\
\text{MathOperand} &\rightarrow \text{ID} \mid \text{VALUE_NUM} \\
\text{MathOp} &\rightarrow + \mid - \mid * \mid / \mid \% \\
\text{Cond} &\rightarrow \text{CondAnd CondOrPrime} \\
\text{CondOrPrime} &\rightarrow || \text{ CondAnd CondOrPrime} \mid \varepsilon \\
\text{CondAnd} &\rightarrow \text{CondRel CondAndPrime} \\
\text{CondAndPrime} &\rightarrow \&\& \text{ CondRel CondAndPrime} \mid \varepsilon \\
\text{CondRel} &\rightarrow \text{MathExpr RelOp MathExpr} \mid \text{CondUnary} \\
\text{CondUnary} &\rightarrow ! \text{ CondUnary} \mid \text{(Cond)} \mid \text{IdVal} \\
\text{IdVal} &\rightarrow \text{ID} \mid \text{VALUE_NUM} \mid \text{VALUE_CHAR} \\
\text{RelOp} &\rightarrow == \mid ! = \mid < \mid > \mid <= \mid >=
\end{aligned}
$$

4. Condiciones Lógicas (Precedencia por Capas):
Para respetar rigurosamente la jerarquía de las operaciones lógicas, las reglas de producción para las condiciones (`Cond`) se diseñaron utilizando un enfoque de niveles de prioridad estratificados:

- **Nivel de Menor Prioridad (OR):** El análisis inicia en `Cond`, obligando a evaluar primero las capas inferiores antes de procesar el operador `||`.
- **Nivel Intermedio (AND):** A través de `CondAnd`, los operadores agrupan sus operandos lógicos antes de que la capa superior (OR) pueda procesarlos.
- **Nivel de Comparaciones (Relacional):** La regla `CondRel` evalúa las expresiones matemáticas separadas por operadores relacionales ($>$, $<$, $==$).
- **Nivel de Máxima Prioridad (NOT y Paréntesis):** La regla `CondUnary` asegura que las negaciones lógicas y las expresiones anidadas entre paréntesis se resuelvan primero.

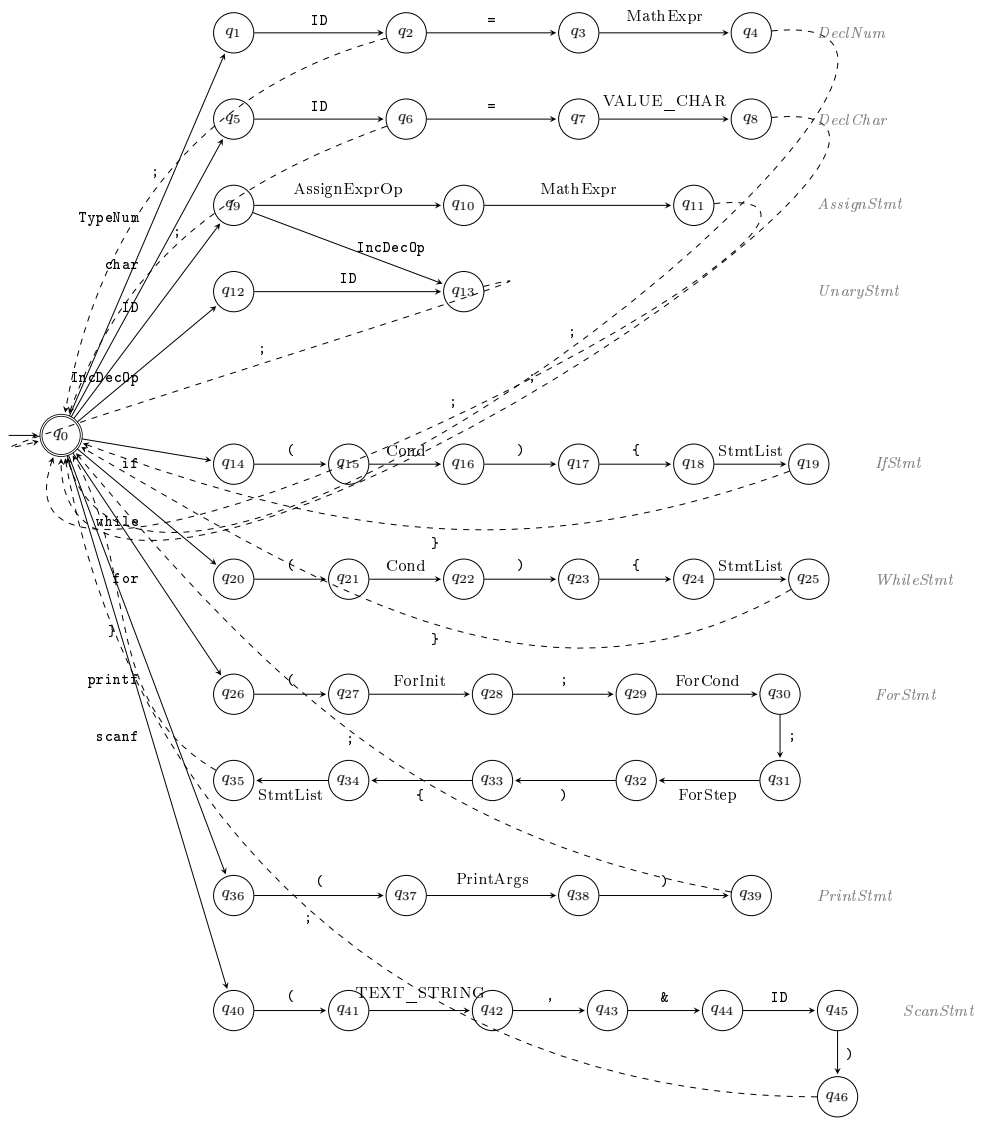
5. Autómata Finito Determinista (DFA) del Analizador Sintáctico Predictivo

Se presenta el autómata de estados finitos determinista que modela el comportamiento del analizador sintáctico predictivo (*parser top-down*) basado en la gramática libre de contexto definida anteriormente. El estado q_0 es el estado inicial y de aceptación (representando la producción Stmt $\rightarrow \varepsilon$). Cada camino desde q_0 corresponde al reconocimiento de un tipo de instrucción; al completarla, el autómata retorna a q_0 para procesar la siguiente instrucción.

Tabla de transiciones por producción:

Producción	Estado origen	Símbolo	Estado destino
DeclNum	q_0	TypeNum	q_1
	q_1	ID	q_2
	q_2	`=`	q_3
	q_3	MathExpr	q_4
	q_4	`;`	q_0
	q_2	`;` (ε)	q_0
DeclChar	q_0	`char`	q_5
	q_5	ID	q_6
	q_6	`=`	q_7
	q_7	VALUE_CHAR	q_8
	q_8	`;`	q_0
	q_6	`;` (ε)	q_0
AssignStmt	q_0	ID	q_9
	q_9	AssignExprOp	q_{10}
	q_{10}	MathExpr	q_{11}
	q_{11}	`;`	q_0
UnaryStmt (prefijo)	q_0	IncDecOp	q_{12}
	q_{12}	ID	q_{13}
	q_{13}	`;`	q_0
UnaryStmt (posfijo)	q_9	IncDecOp	q_{13}
	q_{13}	`;`	q_0
IfStmt	q_0	`if`	q_{14}
	q_{14}	`(`	q_{15}
	q_{15}	Cond	q_{16}
	q_{16}	`)`	q_{17}
	q_{17}	`{`	q_{18}
	q_{18}	StmtList	q_{19}
	q_{19}	`}`	q_0

Producción	Estado origen	Símbolo	Estado destino
WhileStmt	q_0	`while`	q_{20}
	q_{20}	`(`	q_{21}
	q_{21}	Cond	q_{22}
	q_{22}	`)`	q_{23}
	q_{23}	`{`	q_{24}
	q_{24}	StmtList	q_{25}
	q_{25}	`}`	q_0
ForStmt	q_0	`for`	q_{26}
	q_{26}	`(`	q_{27}
	q_{27}	ForInit	q_{28}
	q_{28}	`;`	q_{29}
	q_{29}	ForCond	q_{30}
	q_{30}	`;`	q_{31}
	q_{31}	ForStep	q_{32}
	q_{32}	`)`	q_{33}
	q_{33}	`{`	q_{34}
	q_{34}	StmtList	q_{35}
	q_{35}	`}`	q_0
PrintStmt	q_0	`printf`	q_{36}
	q_{36}	`(`	q_{37}
	q_{37}	PrintArgs	q_{38}
	q_{38}	`)`	q_{39}
	q_{39}	`;`	q_0
ScanStmt	q_0	`scanf`	q_{40}
	q_{40}	`(`	q_{41}
	q_{41}	TEXT_STRING	q_{42}
	q_{42}	`,`	q_{43}
	q_{43}	`&`	q_{44}
	q_{44}	ID	q_{45}
	q_{45}	`)`	q_{46}
	q_{46}	`;`	q_0



Leyenda: Las flechas punteadas (- - -) indican transiciones de retorno al estado q_0.
Las etiquetas en *cursiva gris* identifican la producción correspondiente.

Figure 1: DFA del analizador sintáctico predictivo para la gramática propuesta.

Descripción formal del autómata:

El autómata finito determinista se define formalmente como la 5-tupla $M = (Q, \Sigma, \delta, q_0, F)$ donde:

- $Q = \{q_0, q_1, q_2, \ldots, q_{46}\}$ (47 estados)
- Σ es el conjunto de símbolos de entrada (terminales y no terminales de la gramática)

- $\delta : Q \times \Sigma \to Q$ es la función de transición definida a continuación
- q_0 es el estado inicial
- $F = \{q_0\}$ es el conjunto de estados de aceptación

Observaciones sobre el autómata:

1. **Estado q_0 como estado inicial y de aceptación:** El estado q_0 cumple ambos roles porque la producción Stmt $\to \varepsilon$ permite aceptar la cadena vacía (un programa sin instrucciones). Después de reconocer completamente una instrucción, el autómata siempre regresa a q_0 para iniciar el reconocimiento de la siguiente.
2. **Determinismo por token de anticipación (*lookahead*):** Desde q_0, el primer token de la entrada determina unívocamente cuál camino seguir:
   - `int`, `float`, `double` $\to$ DeclNum (estado q_1)
   - `char` $\to$ DeclChar (estado q_5)
   - `ID` $\to$ AssignStmt o UnaryStmt posfijo (estado q_9)
   - `++`, `--` $\to$ UnaryStmt prefijo (estado q_{12})
   - `if` $\to$ IfStmt (estado q_{14})
   - `while` $\to$ WhileStmt (estado q_{20})
   - `for` $\to$ ForStmt (estado q_{26})
   - `printf` $\to$ PrintStmt (estado q_{36})
   - `scanf` $\to$ ScanStmt (estado q_{40})
3. **Transiciones opcionales (ε):** Los estados q_2 y q_6 poseen dos transiciones por `;`: una directa a q_0 (cuando la variable se declara sin inicializar, es decir AssignOpNum $\to \varepsilon$) y otra que pasa por la asignación antes de llegar al `;` final.
4. **Ambigüedad resuelta en q_9:** Cuando el primer token es `ID`, el autómata transita a q_9. Desde ahí, el segundo token resuelve la ambigüedad:
   - Si es un *AssignExprOp* (`=`, `+=`, `-=`, `/=`) $\to$ se sigue el camino de AssignStmt.
   - Si es un *IncDecOp* (`++`, `--`) $\to$ se sigue el camino de UnaryStmt posfijo.
5. **Símbolos no terminales como transiciones:** Las etiquetas como *Cond*, *StmtList*, *MathExpr*, *PrintArgs*, *ForInit*, *ForCond* y *ForStep* representan llamadas recursivas al parser predictivo, donde cada no terminal se expande según sus propias reglas de producción definidas en la gramática.

3.3 Entrada/Salida y Ejecución

El analizador léxico desarrollado soporta dos modos de operación para entradas en lenguaje C, lectura desde archivo y lectura desde teclado.

1. **Lectura desde archivo.** En este modo, el usuario debe colocar el archivo en la carpeta inputs y el sistema lo lee usando codificación UTF-8 y posteriormente procesa el contenido línea por línea y al final del análisis léxico, los resultados pueden ser exportados hacia la carpeta outputs.

2. **Lectura desde teclado.** El analizador permite ingresar código fuente directamente desde la terminal. La captura finaliza cuando el usuario introduce el símbolo terminal `$`, el cual actúa como delimitador de fin de entrada.

Durante la ejecución, el sistema realiza las siguientes operaciones principales:

- Tokenización del código fuente mediante el lexer de C proporcionado por la biblioteca *Pygments*.
- Clasificación de cada token conforme a las categorías definidas en el curso (Keyword, Identifier, Operator, Punctuator, Constant y Literal).
- Eliminación de tokens irrelevantes como espacios en blanco o saltos de línea que no aportan información sintáctica.
- Agrupación y conteo total de tokens por categoría.
- Visualización opcional del código fuente con resaltado de tokens en colores y exportación de resultados a un archivo de texto.

Cabe mencionar que el caso particular del carácter `&` utilizado en la función `scanf()`, así como ciertos delimitadores como paréntesis, llaves, comas y punto y coma se trataron explícitamente como punctuators debido a que la biblioteca utilizada no los categoriza de forma consistente.

3.4 Relación con la CFG propuesta

El diseño del analizador léxico se realizó tomando como referencia la gramática libre de contexto definida para el lenguaje C reducido considerado en este proyecto. Nuestra CFG contempla instrucciones como declaraciones de variables numéricas y de carácter, asignaciones, estructuras de control (`if`, `while`, `for`), operaciones unarias de incremento y decremento, así como llamadas a funciones de entrada/salida (`printf` y `scanf`) **cfglexer**.
También se definieron reglas de producción para expresiones matemáticas y condiciones lógicas respetando la precedencia de operadores mediante un enfoque de derivación por niveles, donde los operadores lógicos OR presentan la menor prioridad, seguidos por AND, operadores relacionales y finalmente operadores unarios como la negación lógica y el uso de paréntesis.

3.5 Pruebas realizadas

Para validar nuestro analizador léxico, se propusieron un par de pruebas utilizando fragmentos de código en lenguaje C que incluyen declaraciones, expresiones aritméticas, llamadas a funciones de entrada/salida y estructuras de control contempladas en la CFG propuesta.

- **Código 1: Declaraciones y expresiones aritméticas**

  En esta prueba se analizó un programa que incluye declaraciones de variables numéricas, una asignación con expresión aritmética y una llamada a la función `printf()`. El analizador léxico identificó correctamente palabras clave, identificadores, operadores de asignación y operadores aritméticos, así como constantes numéricas y delimitadores del lenguaje.

  De igual forma, se verificó el reconocimiento adecuado del literal de cadena utilizado en la función de salida, el cual fue agrupado como un único token. El resultado del análisis permitió obtener el conteo total de tokens clasificados por categoría, validando la correcta tokenización de expresiones aritméticas y sentencias de impresión.

- **Código 2: Entrada de datos y estructura iterativa**

  En esta prueba se evaluó un programa que realiza lectura de datos mediante `scanf()` y emplea una estructura iterativa `for`. Durante el análisis se reconocieron correctamente las palabras clave asociadas al ciclo, los operadores relacionales y de incremento, así como los delimitadores correspondientes.

  También se verificó la clasificación del símbolo especial `&` utilizado para la lectura por referencia y la correcta identificación del literal de cadena dentro de la función `printf()`. Los resultados confirmaron que el analizador puede procesar adecuadamente instrucciones iterativas y llamadas a funciones de entrada/salida, generando la tabla y el conteo agrupado de tokens.

4 Resultados

Mediante capturas de pantalla y una breve descripción seguida de la captura se presentan los resultados finales de su aplicación.

5 Conclusiones

Se presenta un análisis de los resultados obtenidos, donde se destaca la importancia de la aplicación de los conceptos teóricos para resolver el problema. Es importante resaltar lo siguiente:

- No es describir si les gustó la actividad o no.

- No es decir que se obtuvo de la práctica.
- No es describir lo que fue difícil.

Referencias

[1] A. Holub, *Compiler Design in C*. Prentice-Hall, 1990, ISBN: 0-13-155045-4. [Online]. Available: https://holub.com/goodies/compiler/compilerDesignInC.pdf

[2] Pygments Contributors. "Builtin tokens," Accessed: Mar. 13, 2026. [Online]. Available: https://pygments.org/docs/tokens/